

Prevalence of Strokes with Chronic Diseases in Elderly, German Patients

Se Rang Seo

May 2026

Abstract

This project analyzes 10,000 patients from a larger IQVIA dataset (originally 840,319 patients) to identify correlation between prevalence of chronic diseases and stroke risks by building a predictive model to identify stroke-prone patients. Among all chronic diseases recorded, hypertension was the most prevalent at 67.12%, while stroke was the least common at just 6.27%. Correlation analysis showed that hypertension, coronary heart disease (CHD), and chronic kidney disease (CKD) has the strongest associations with stroke. Five supervised machine learning models were tested: logistic regression, SVM, KNN, XGBoost, and random forest. SVM performed best at detecting actual stroke patients, achieving a recall score of 77%—correctly identifying 145 out of 188 stroke cases in the test set of 3,000 patients. Logistic regression delivered the highest AUC score at 0.714, meaning it would likely generalize better to new datasets. KNN performed the worst for stroke detection, catching only 5% of stroke patients (9 out of 188). For unsupervised learning, three clustering methods (k-means, hierarchical, and Gaussian mixture) were used to group patients by risk level to stroke. Hierarchical clustering achieved the highest silhouette score (0.216) and separated patients into three distinct risk tiers, with GMM clustering showing the the highest-risk cluster with stroke prevalence of 11.7%. These results suggest that SVM is a strong choice for maximizing stroke detection sensitivity, while GMM clustering offers a practical way to group patients into low, medium, and high-risk groups for more efficient medical resource allocation.

Contents

1	Introduction	3
2	Methods	3
2.1	Feature Engineer and Data Analysis	3
2.2	Supervised Learning	6
2.2.1	Logistic Regression	6
2.2.2	Support Vector Machine (SVM)	6
2.2.3	K-Nearest Neighbors (KNN)	7
2.2.4	XGBoost	7
2.2.5	Random Forest	8
3	Supervised Learning Result	8
4	Unsupervised Learning	11
5	Conclusion	14

1 Introduction

The dataset was originally sourced from IQVIA in 2016, a private health research company, and was published by Prof. Dr. Karel Kostev. While his original research included 840,319 patients, only 10,000 were made publicly available since it was classified research done by the company.

The dataset comprises 12 parameters: gender, age, hypertension (chronic high blood pressure), heart failure (reduced pumping ability of the heart), CKD (chronic kidney disease; progressive loss of kidney function), lipid metabolism disorders (abnormal cholesterol or fat levels), osteoporosis (weakening of bones), COPD (chronic obstructive pulmonary disease; progressive lung damage), CHD (coronary heart disease; narrowed arteries of the heart), stroke (acute neurological event due to disrupted blood flow to the brain), and cancer (uncontrolled cell growth). All diseases are classified as chronic diseases (CDs).

Identifying patients who are prone to experience a stroke, either for the first time or as a recurrent event, is of critical importance. Through this machine learning analysis, we aim to explore any correlations between stroke, age, gender, and the listed CDs. To achieve this, I will:

1. Build a predictive machine learning model that identifies stroke-prone patients based on age and the presence of chronic diseases. This will be done using supervised learning with various classification algorithms.
2. Classify patients into distinct risk levels using unsupervised clustering, ensuring optimal allocation of medical attention and resources.

2 Methods

2.1 Feature Engineer and Data Analysis

The only feature engineering required is dropping patient ID. There were no NA instances in the dataset. To create a correlation matrix limited to binary variables, I excluded age.

patient	male	gender	age	Hypertension	CHD	Lipid_disorder	Stroke	Heart_failure	Cancer	Diabetes	COPD	Osteoporosis	Chronic_kidney_disease
0	1	1	68	1	0	1	0	0	0	0	0	0	0
1	2	0	78	1	0	0	0	0	0	0	0	1	0
2	3	0	78	0	0	0	0	0	0	0	0	0	0
3	4	0	92	0	0	1	0	0	0	0	0	1	0
4	5	1	71	1	0	1	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
9995	9996	1	80	1	1	1	0	0	0	0	0	0	0
9996	9997	0	67	1	0	1	0	0	0	0	0	0	0
9997	9998	0	90	0	0	0	0	0	0	0	0	0	0
9998	9999	0	67	0	0	0	0	0	1	0	0	0	0
9999	10000	0	69	0	0	1	0	0	0	0	0	0	0

Figure 1: This is the head of the dataset with 13 parameters and 10k rows. I will drop patient number in the first column.

In Figure 2, we can see some correlations between strokes and other diseases. We can also see a slightly higher weight toward male patients (0.04) than female patients. After running a correlation algorithm with Python to identify the most highly correlated pairs, the top ten results are shown in Table 1.

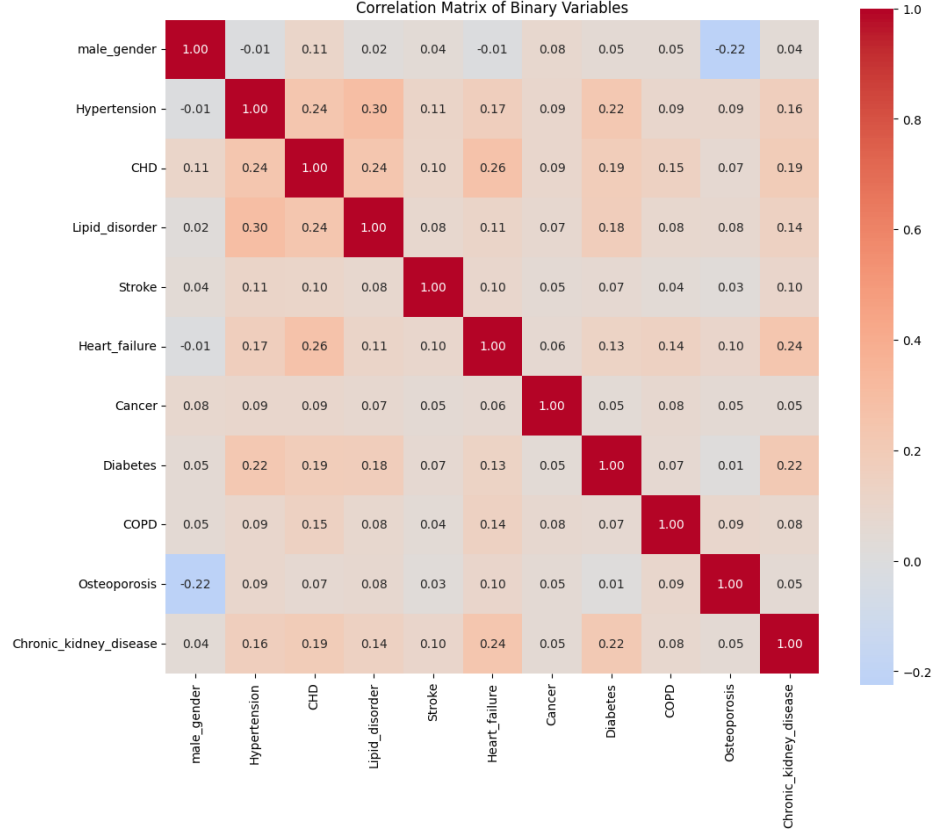


Figure 2: This is a correlation matrix pairing features with each other to find strong and weak connections.

This is a correlation matrix of each binary parameter. We can read this plot by looking horizontally on the row of stroke. This shows that hypertension, CHD, and CKD have the highest stroke risk. The 3 weakest correlation with strokes are osteoporosis, COPD and cancer. The negative on the male column/row means it's more correlated to females.

Table 1: Highest Correlations Among Chronic Diseases (Top 10)

Variable 1	Variable 2	Correlation
Hypertension	Lipid disorder	0.296 536
CHD	Heart failure	0.259 815
CHD	Lipid disorder	0.243 281
Hypertension	CHD	0.240 682
Heart failure	Chronic kidney disease	0.236 093
Hypertension	Diabetes	0.222 711
Diabetes	Chronic kidney disease	0.218 944
CHD	Chronic kidney disease	0.191 135
CHD	Diabetes	0.186 092
Lipid disorder	Diabetes	0.178 401

We see that from this table that hypertension, lipid disorder, and chronic heart disease are strongly correlated.

Since the age is the only feature with whole numbers while elsewhere were binaries, I was able to create a violin plot studying couple of CD's with age.

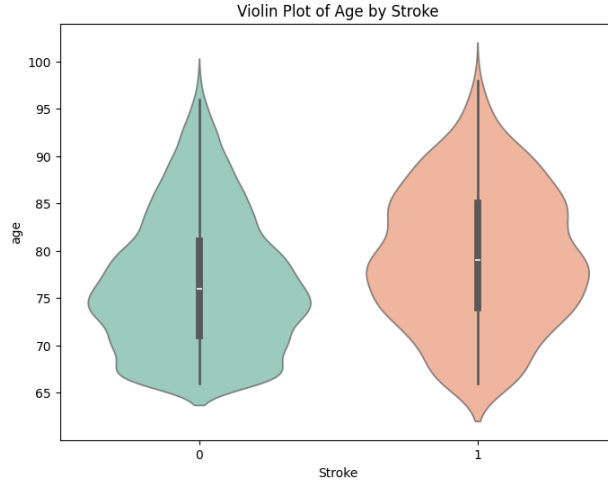


Figure 3: This is a violin plot between age and stroke.

We can see from Figure 3 that most people with strokes are between age 75 and 79 whereas the most people without strokes are between 73 and 74. And, more people tend to experience stroke much more than not as they age beyond 85. The violin plot shows that stroke patients are concentrated in the 75–79 age range, while the non-stroke group has a broader, more symmetric age distribution. The narrowing of the non-stroke violin at older ages suggests that stroke becomes relatively more common after age 85.

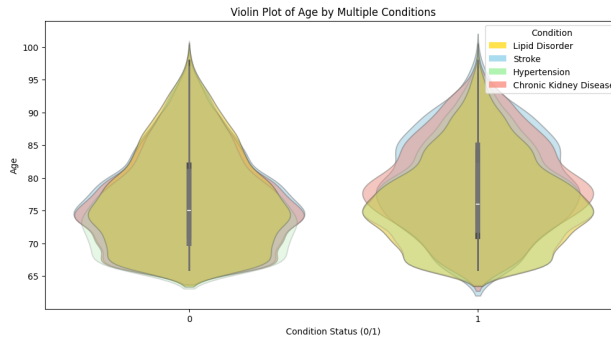


Figure 4: This violin plot has 4 CD overlapped on each other. The front yellow is the mixture of lipid disorder and hypertension.

We can also see from Figure 4 that compared to other CD, strokes are prevalent even among patients past age 85 similar to CKD. Lipid disorder is most prevalent at age 74 and chronic kidney disease is most common in age 76.

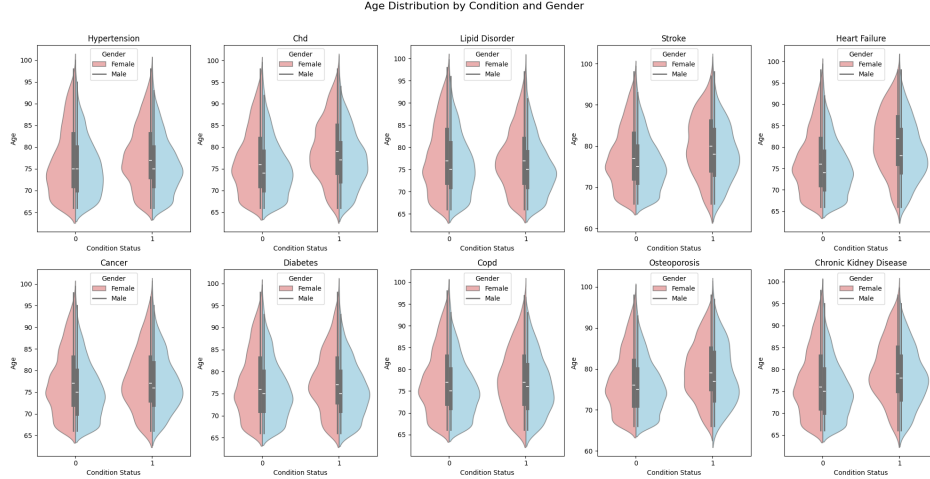


Figure 5: Violin plot for each CD split based on gender. 0 means they don't have it and 1 means they do.

Figure 5 shows individual CD where 0 is negative and 1 is positive. For most plots, the violin shape thins out towards older ages beyond 80, but stroke's plot stays relatively unchanged as the age gets higher. We can also see the balance between male and female. A larger blue area indicates that the age distribution for males is more spread out or has a different density shape compared to females, not necessarily a larger number of males.

In general, we can see from correlation matrix and violin plots that there is strong correlation with CHD, CKD, heart failure, and lipid disorder. The violin plot also tells us that most patients are ages between 70 and 80.

2.2 Supervised Learning

To maximize the identification of stroke patients, all models were optimized for recall using grid search with 5-fold cross-validation. Class weights were set to 'balanced' to address dataset imbalance. Features were standardized using StandardScaler, and data were split 70/30 into training and test sets. High recall was prioritized over precision, as false negatives (missed strokes) carry greater clinical risk than false positives, which can be resolved through follow-up testing.

2.2.1 Logistic Regression

Our first algorithm is the logistic regression, It can take in any variable type and output binaries. The outcome will be similar to a sigmoid graph between 0 and 1 representing the probability of classification. The hyperparameter that I explored were C variable and solver. C variable is a regularization constant where higher the C means weaker regularization. Solvers are the algorithm of logistic regression. Grid search is shown below:

```
lr_param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'lbfgs', 'saga']
}
```

There are a total of 75 fits and took 4.2 seconds to run.

2.2.2 Support Vector Machine (SVM)

Secondly, SVM takes in parameters and increases its dimensions as needed to encapsulate all the features. The hyperparameter that I tuned were the C variable, γ , and kernel. The C variable is regularization constant like logistic regression where higher the C value means the algorithm is trying to classify every single data

point leading to overfitting. Another word for γ is the kernel coefficient. Higher γ means a single data point influences the next point that is very close. Lower γ allows for a longer distance influence. Lastly, kernel is the algorithm of SVM. Rbf (radial basis function) maps the dataset into infinite dimensions identifying other points that are similar to the current data point. Sigmoid is similar to the sigmoid activation function from neural network making it act like a classifying, miniature neural network.

The hyperparameters that I explored were C, kernel, and gamma. C is a regularization constant where higher C means weaker regularization. Kernel specifies the type of kernel function to be used (linear, rbf, or poly). Gamma defines the influence of a single training example, with higher gamma meaning closer influence. Grid search is shown below:

```
svm_param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'kernel': ['linear', 'rbf', 'poly'],
    'gamma': ['scale', 'auto']
}
```

There are a total of 250 fits ($5 \times 3 \times 2 = 30$ combinations $\times$ 5-fold CV) and took 1 minute and 41 seconds to run.

2.2.3 K-Nearest Neighbors (KNN)

Third, KNN takes a data point and detects all the points around it in a distance. I decided to tune with it a bit more since it ran much faster. The hyperparameters that I explored were: number of neighbors, weights, metric, algorithm, and leaf size. N numbers is how many patients the machine is looking around from a single data point. Weights dictate how much the votes count on each neighbor. Metric measures the similarities with neighbors. Algorithm is how the machine finds the neighbors. Leaf size only affects the ball tree and kd tree algorithms which affects speed of the run. The grid search is shown below:

```
knn_param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski'],
    'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
    'leaf_size': [10, 20, 30, 40, 50]
}
```

The total fit is 1260 ($5 \times 2 \times 3 \times 4 \times 5 = 600$ combinations $\times$ 2-fold CV) and took 13.9 seconds to run.

2.2.4 XGBoost

XGBoost (Extreme Gradient Boosting) builds an ensemble of decision trees sequentially, where each tree learns from the previous one's errors—similar to gradient descent. Hyperparameters tuned include: number of estimators (tree count), tree depth (higher depth increases overfitting risk), learning rate (step size), subsample (fraction of patients per tree), colsample (fraction of features per tree), minimum child weight (prevents overly specific branches), gamma (minimum loss reduction for splitting), and L1/L2 regularization (α , λ) to penalize model complexity. The grid search is shown below:

```
xgb_param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.3],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'min_child_weight': [1, 3, 5],
    'gamma': [0, 0.1, 0.2],
    'reg_alpha': [0, 0.1, 1],
}
```

```

    'reg_lambda': [0, 0.1, 1]
}

```

With 2-fold cross-validation, this resulted in 2880 total fits ($3^9 = 19683$ combinations would be too large, so a randomized search with 1440 combinations $\times 2$ folds was used) and completed in 27.1 seconds.

2.2.5 Random Forest

Lastly, I used the random forest decision tree ensemble. It's a forest of decision trees and measures the most prominent result through averaging. The hyperparameters I used were estimator, max depth, sample split, sample leaf, and class weight. Estimator and depth are the same as XGBoost. Sample split and leaf are different in that split is the decision making yes or no, but the leaf is the final prediction representing what the machine learned. The grid search is shown below:

```

rf_param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 7, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'class_weight': ['balanced', 'balanced_subsample', None]
}

```

Total fit is 270 ($3 \times 4 \times 3 \times 3 \times 3 = 324$ combinations, but with None depth and some pruning, it resulted in 270 fits with 1-fold CV) and took 9.5 seconds to run.

3 Supervised Learning Result

After optimization, SVM achieved the highest recall at 77% (identifying 77% of stroke patients; missing 23%), a clinically meaningful result despite being imperfect. KNN performed worst, followed by Random Forest, though both had higher accuracy. As expected, precision was low across models since recall and precision are inversely related. For overall performance, logistic regression had the highest AUC (0.714) compared to SVM (0.660), indicating better generalizability to different datasets. All AUC scores averaged 0.65, with KNN lowest at 0.537. Additionally, the current analysis used only a fraction of the available data—the full dataset is $80\times$ larger—suggesting potential for further improvement.

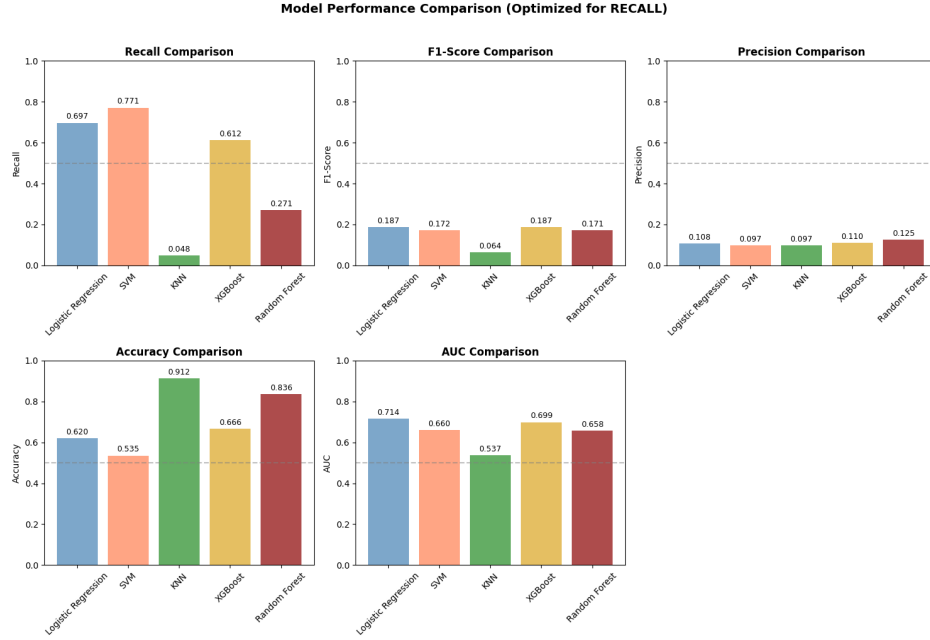


Figure 6

The ROC curve in Figure 7 shows the ratio between true and false positives rates. KNN's ROC curve rises sharply near 0.15, then follows a near-diagonal line, indicating poor discriminative ability beyond low false-positive rates. SVM has a steeper spike between 0.4 and 0.5. Logistic regression and XGBoost has a more overall curve towards the outside of the plots creating more of a smoother elbow compared to SVM's abrupt increase. Random forest catches up to the other curves towards the end, but it performs mediocly consistently through the curve.

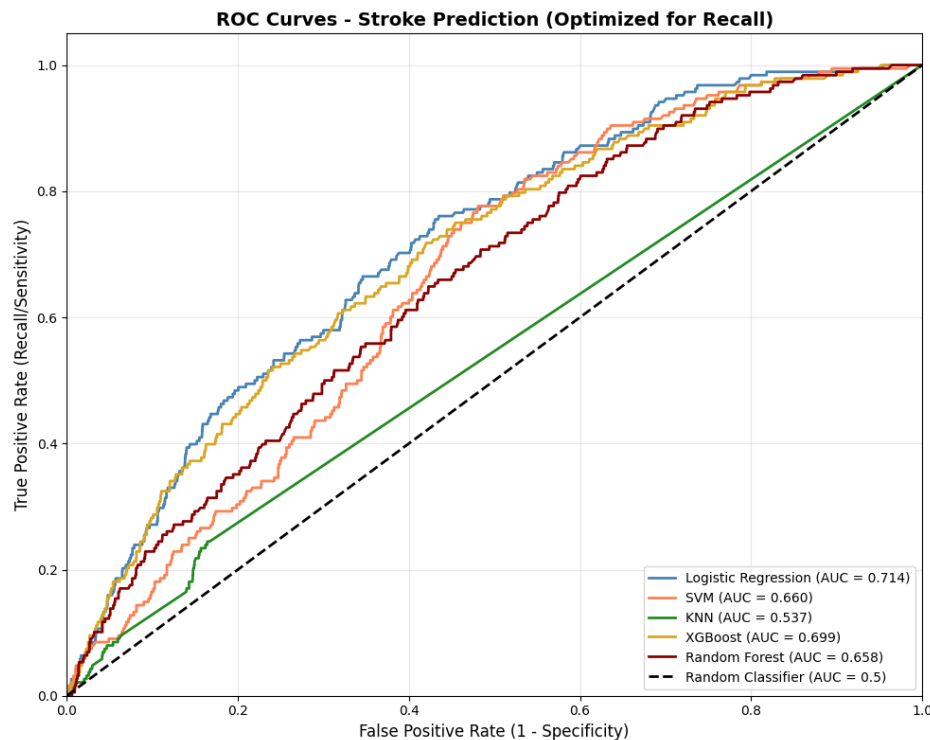


Figure 7: The ideal curve is elbowing toward top left. Straight line like KNN means the classification is simply randomized 50:50.

Lastly, looking at the confusion matrix in Figure 8, only the top row is used to calculate recall: $\text{Recall} = \frac{\text{top left}}{\text{top left} + \text{top right}}$. The total number of patients used for testing is 3,000, so out of 3,000 patients, from SVM, 145 patients were correctly identified to have stroke while missing 43 patients. Even though we missed 23%, it is much better than a model like KNN where it identified only 9 while missing 179. With 1351 false positives, the model flagged many non-stroke patients as potential stroke cases, leading to a low precision of around 10%.

KNN excelled in identifying patients who didn't have stroke even though it did poorly in completing all the patients prone to stroke. In that way, random forest is the least effective in this classification because it does not excel in either categories.

XGBoost and logistic regression both had similar results. But, while logistic regression did better in grouping patients with strokes.

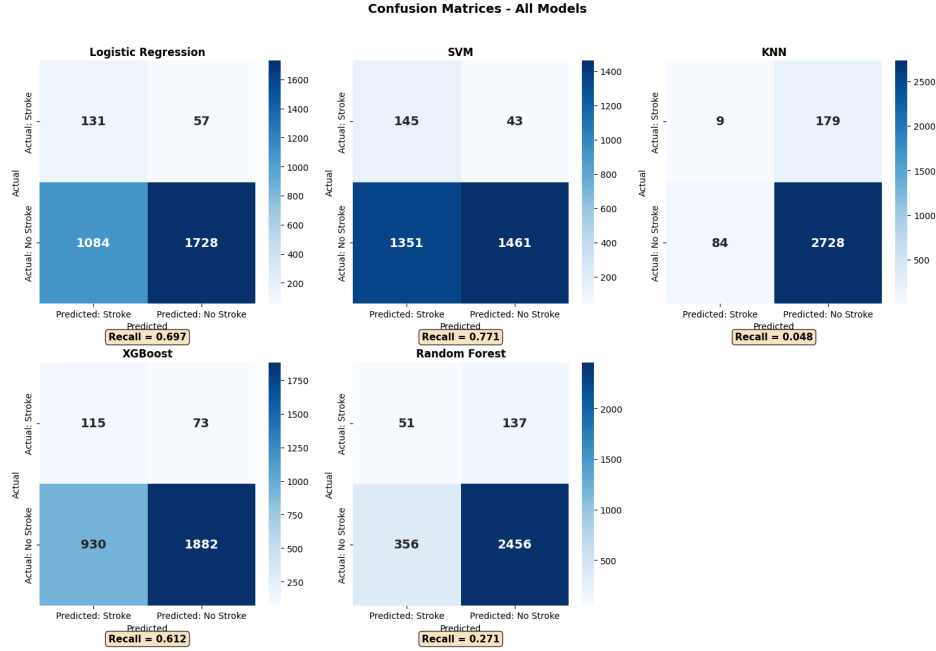


Figure 8: Confusion matrix of all of the supervised learning algorithms.

In all, out of the 5 models, SVM did the best in recall while logistic did the best in AUC. KNN despite the worst performance in optimizing recall and precision, it performed best in identifying patients without strokes. Among all models, KNN performed worst in identifying stroke patients. Random Forest also showed relatively weak performance, with moderate recall and precision.

4 Unsupervised Learning

For this portion, I used k-means clustering, hierarchical clustering, and Gaussian mixture model. to create a tiered group of patients that are more prone to strokes than others based on the machine learning. Like with supervised learning, all of the data are scaled using StandardScaler before fitting and split .7 (train) to .3 (test) sets.

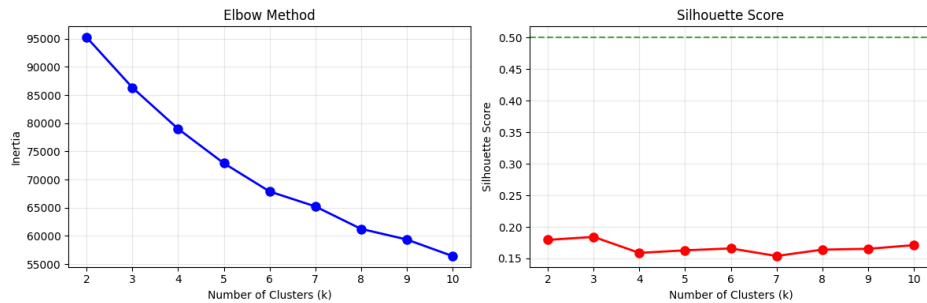


Figure 9: There were no strict steepness occurring in the elbow plot, so it deemed useless. Silhouette plot allowed us to set clusters to 3 with highest score of 0.216.

The methods for all of these were to find the optimal silhouette value before running to find the best number of clusters and correlation. Silhouette score measures how well a point fits into each cluster compared to other clusters. The elbow method is only useful if there is a steepness to the cluster, but in my graph there wasn't any. So, I had to use silhouette plot to identify the best number of clusters which is 3. After

running the cluster algorithms, the results shown in Figure 10 shows the PCA dimension reduction plot of the dataset into two components.

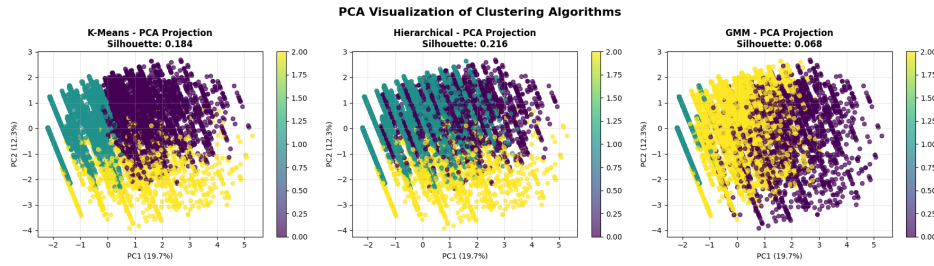


Figure 10: This graph only shows 33% of the total dataset. There are some similarities between first and second plots. Overall data point locations are similar.

PCA results 19.7% and 12.3% of the dataset which means that the algorithm is able to narrow down the correlated features into 2 components where the PCA graph is showing only 33% of the dataset. The silhouette score for k-means method is 0.184, hierarchical method is 0.216, and GMM is 0.068. K-means and hierarchical plot group show very similar pattern while GMM has rotated yellow and purple cluster comparatively.

The t-sne plots shown in Figure 11 shows more isolated clusters of the patients. We look for overlaps and similarities between models. All three plots have the purple cluster at the right, yellow cluster at the bottom right, and the teal cluster at the majority left.

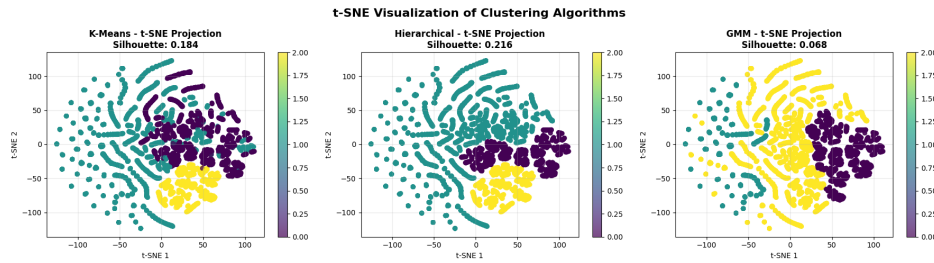


Figure 11: The clustering of each color is very similar in plot 1 and 2. The color spread is very similar even with the third plot.

We can visualize the silhouette plot to see how the clusters and the silhouette coefficients interact. Low silhouette scores are mostly below .50, and that means the data plots are mixed up too much where it is extremely difficult to separate them without convoluting another.

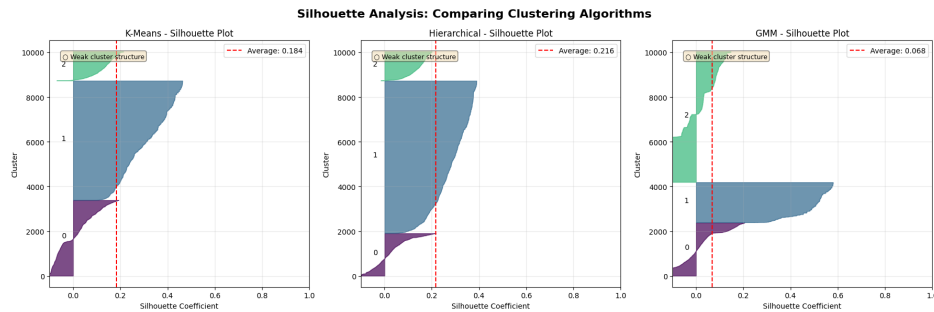


Figure 12: The silhouette scores are measure based on each model. Hierarchical model has the highest score of 0.216.

In clustering, silhouette score tells us the best model. Even though most of our scores are very low, hierarchical clustering gives us the highest silhouette score. We can see from Figure 13, that the clusters 0 and 3 are much more prone to strokes than the 1st cluster in all three models. This tells us that we can group these clusters into 3 tiers: high risk, medium risk, and low risk stroke patients. Even though hierarchical clustering had .216 as silhouette score while GMM has 0.068, GMM has the best result in separating stroke-prone patients.

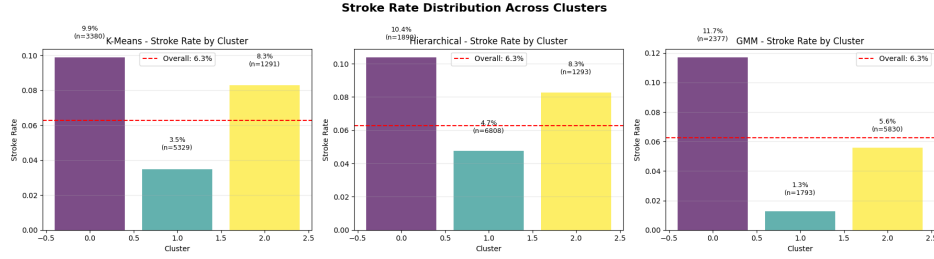


Figure 13: This plot shows the percentage of stroke patients in each cluster for each model.

Figure 14 shows us statistics of each cluster. We see that each cluster has similar age of 77-79. Cluster 2 shows the highest mean osteoporosis prevalence, while Cluster 1 lacks chronic kidney disease (CKD), which is strongly correlated with stroke. This aligns with earlier correlation findings. Diabetes and CKD is prevalent in clusters 0 and 2. Cluster 2 has the least amount of females which signifies higher ratio of males with strokes.

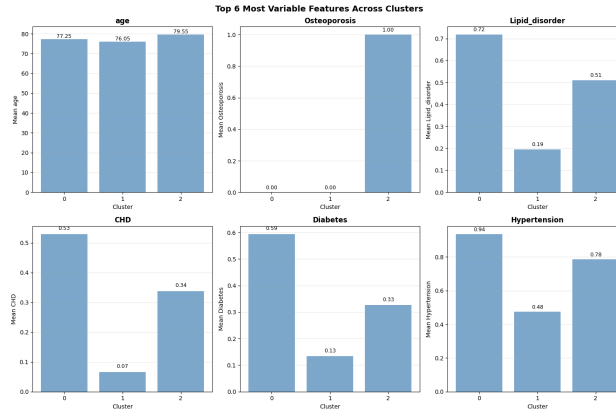


Figure 14: This shows the prevalence of CD and age for each cluster.

Dropping age increased the silhouette score slightly to 0.239 and allowed 10 clusters, though the low score still suggests weak cluster separation. This approach sacrifices interpretability for marginal gain.

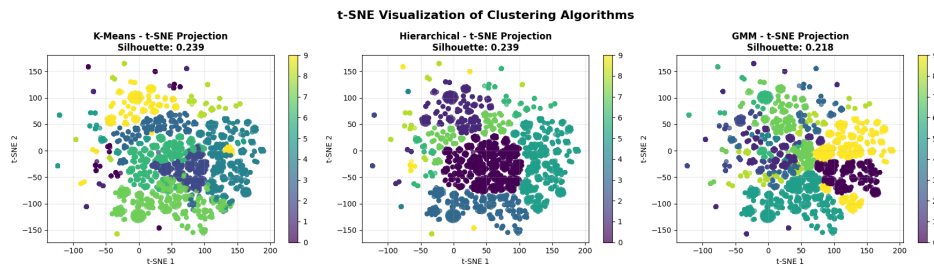


Figure 15: This is t-sne plot without age. Although we lose information about age, it created a more separated and somewhat bubbly plot.

In summary, unsupervised learning provides a way to group patients into three clusters: Cluster 0 (highest risk), Cluster 1 (lowest risk), and Cluster 2 (medium risk) for stroke. This allows hospitals and medical facilities to identify and group patients based on their risk tier, guiding them to allocate assistance efficiently and effectively.

5 Conclusion

The original dataset has more than 800k instances, but with only 10k available, the models still gave plausible results. SVM achieved a recall of 77%, and hierarchical clustering produced three groups, with the highest-risk cluster showing a stroke prevalence of 11.7%, compared to the overall dataset prevalence of 6.27%. From the correlation matrix, we can also group patients with CHD, CKD, and hypertension due to their strong correlation with strokes.

There is much more to be done: tuning more hyperparameter, improving model fit, and performing feature engineering by studying learning curves. There were also a lot of key information missing such as body weight, blood pressure, previous incidents, and so on that could have been useful in running these algorithms.

For future plans, I want to explore why GMM gives better clustering of stroke patients when hierarchical method has much higher silhouette score. Furthermore, instead of studying stroke-prone patients, we can focus on cancer patients or other chronic diseases, apply more classifiers and clustering methods, or search for a similar dataset that includes chronic diseases as parameters and is more up to date, ideally near 2026.